

A READER'S GUIDE

Video Intake & Blender VSE Pipeline

From a folder of mismatched footage
to an edit-ready Blender project.

Generated from the repository README and source · for evening reading

Contents

Part I · The Project README (verbatim)

Part II · What It Is & Why It Exists

Part III · How It Is Structured

Part IV · How Each Stage Works

Part V · How It Was Made: Design Decisions

Part VI · How to Use It: Full Walkthrough

Part VII · Glossary

Appendix A · The blender-km-macros README (verbatim)

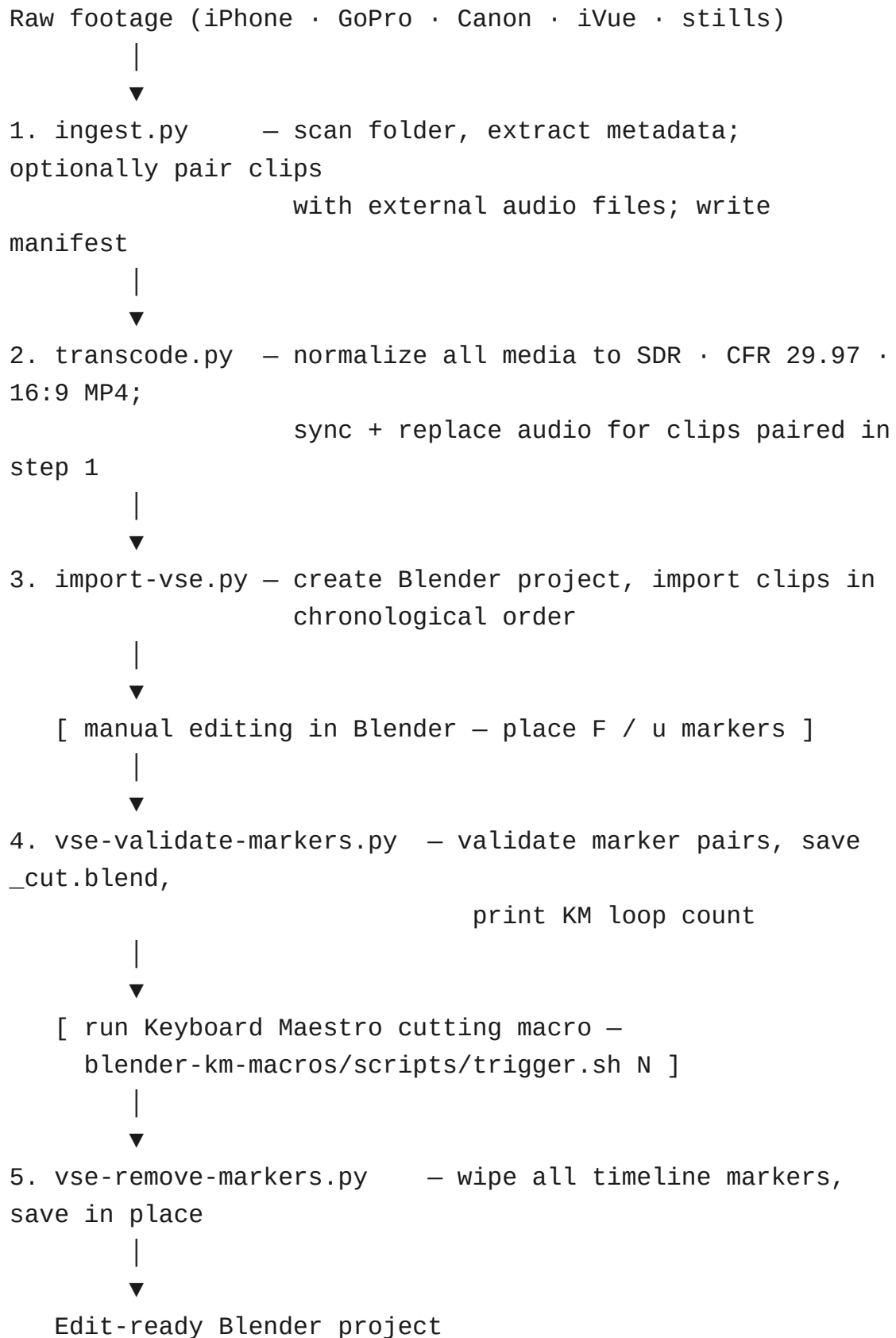
Part I - The Project README (verbatim)

Reproduced exactly as it appears in the repository.

Video Intake & Blender VSE Pipeline

Automates the journey from raw multi-camera footage (iPhone, GoPro, Canon Vixia / 7D, iVue Rincon, stills) to an edit-ready Blender VSE project. Run the four scripts in order, then do your manual editing in Blender, then use the two marker scripts to prep for and clean up after the Keyboard Maestro cutting macro.

Pipeline Overview



Dependencies

Tool	Purpose	Install
ffmpeg + ffprobe	transcode, probe metadata	brew install ffmpeg
avconvert	iPhone HDR → SDR (macOS only)	built-in at /usr/bin/avconvert
ImageMagick convert + identify	image padding, dimension reads	brew install imagemagick
exiftool	EXIF orientation + timestamps on images	brew install exiftool
blender	headless VSE import + marker scripts	blender.org or brew install --cask blender
Keyboard Maestro	run the clip-cutting macro (step 4.5) — macOS only	keyboardmaestro.com · setup in blender-km-macros/
audio-offset- finder	sync external audio to video (optional — only needed if you use external audio files)	pip install audio- offset-finder

Linux Mint: replace avconvert with ffmpeg zscale tone-map (handled automatically). exiftool via sudo apt install libimage-exiftool-perl.

Scripts

1 — ingest.py

Scans a footage folder recursively, extracts metadata via `ffprobe` / `exiftool`, and writes three output files to `<input_dir>/_ingest/`:

- `manifest.json` — machine-readable, consumed by `transcode.py`
- `clips_ordered.txt` — human-readable chronological list
- `ingest_report.md` — counts, flags (HDR / VFR / vertical / missing timestamps), timeline gaps

What it detects per file: source (GoPro / iPhone / unknown), media type, orientation, dimensions, duration, FPS, HDR, VFR, creation timestamp.

External audio pairing. After scanning, `ingest.py` asks whether any video clips have a separate, higher-quality audio file (iPhone Voice Memo, dedicated recorder, etc.). If yes, you drag-and-drop each video clip and its matching audio file. The pairing is written to `manifest.json` as an `external_audio` field — no heavy processing happens at ingest time. The actual sync and audio replacement runs in `transcode.py` (step 2) using `audio-offset-finder` to align the tracks automatically.

Timezone normalization. iPhone and GoPro write `creation_time` in true UTC. Other cameras — Canon Vixia, Canon 7D, iVue Rincon — write naive *local* wall-clock time but still label it `z`, so left alone they sort hours away from the Apple/GoPro clips. Ingest detects clips with no UTC proof, reads the real local offset from an iPhone/GoPro clip in the same batch (e.g. `-0400`), and rewrites the naive clips to true UTC so every clip shares one clock. If there is no Apple/GoPro clip in the batch it falls back to this machine's timezone (and warns) — use `--local-offset` to set it explicitly. Any per-camera embedded timezone tag is deliberately ignored, since it is often misconfigured (the Vixia reports `-05:00` even when shooting in `-04:00`). Displayed times in the reports are local wall-clock.

```
python3 ingest.py /path/to/footage
python3 ingest.py /path/to/footage --output
/path/to/output_dir
python3 ingest.py /path/to/footage --local-offset -04:00
# footage shot in a zone with no Apple/GoPro clip
python3 ingest.py          # prompted — supports drag-and-
drop
```

Flags / options:

Flag	Description
- - output , -o	Custom output directory (default: <input_dir>/_ingest)
- - local - offset	UTC offset of the footage's local time, e.g. -04:00 . Overrides offset detection for naive-local cameras (Canon, iVue). Default: read from an iPhone/GoPro clip, else this machine's timezone.

2 — transcode.py

Reads `_ingest/manifest.json` and normalizes every file. All output lands in `_ingest/transcoded/`. Already-transcoded files are skipped, so re-runs are safe.

What it does per file type:

Input	Transform
Still image	6-second slow-zoom MP4: 1s static lead-in + 5s Ken Burns zoom (libx264 · 29.97 fps · 1920×1080)
iPhone HDR video	avconvert PresetAppleM4V1080pHD → SDR .m4v, then ffmpeg for VFR/vertical/remux
Other HDR video	ffmpeg zscale tone-map → SDR
VFR video	ffmpeg -fps_mode cfr → CFR 29.97
Vertical video	ffmpeg blurred 16:9 letterbox (portrait centred on blurred+darkened background)
GoPro / clean video	stream-copy (fast, no re-encode)

Audio is normalized to **-16 LUFS / -1.5 dBTP** via `loudnorm` on all re-encoded clips.

External audio replacement. Clips that have an `external_audio` entry in the manifest (set during ingest) are processed with `audio-offset-finder` to compute the sync offset, then `ffmpeg` replaces the native audio track with the external file. All standard transforms (HDR → SDR, VFR → CFR, vertical letterbox) still apply alongside the audio swap.

On completion writes `_ingest/transcoded/manifest_transcoded.json` — the input for

import-vse.py.

```
python3 transcode.py /path/to/project_folder
python3 transcode.py /path/to/project_folder --dry-run
python3 transcode.py          # prompted – supports drag-
and-drop
```

Flags / options:

Flag	Description
--output , -o	Custom output directory
--fps	Target CFR frame rate (default: 29.97)
--dry-run	Print what would run, write nothing
--no-loudnorm	Skip audio loudness normalization

3 — import-vse.py

Reads manifest_transcoded.json, prompts for project name and resolution, then launches Blender headlessly to create a .blend file with all clips placed on the VSE timeline in chronological order. Applies the Video Editing workspace layout and writes blender_import.log.

```
python3 import-vse.py /path/to/project_folder
python3 import-vse.py /path/to/project_folder --name
my_project
python3 import-vse.py          # prompted – supports drag-
and-drop
```

Flags / options:

Flag	Description
--name, -n	Project and .blend filename
--blender	Path to Blender executable (auto-detected if omitted)
--dry-run	Show import order and Blender command without running

Prompted interactively: project name, resolution (1080p or 4K), confirmation of clip order.

4 — vse-validate-markers.py

Opens a .blend file in Blender headlessly, reads all timeline markers, and validates them as F / u pairs before you run the Keyboard Maestro cutting macro.

Marker convention: markers must come in pairs — the start of a keep-range is named F1, F2, etc.; the matching end is named exactly u. Each pair defines one segment to retain.

What it checks:

- Even number of markers (every start has an end)
- Alternating F... / u naming
- No overlapping ranges

On success: saves a `_cut.blend` copy of your project and prints the number of KM macro loops needed.

```
python3 vse-validate-markers.py /path/to/project.blend
python3 vse-validate-markers.py           # prompted –
supports drag-and-drop
```

4.5 — Keyboard Maestro cutting macro (`blender-km-macros/`)

Between validating markers and removing them, run the Keyboard Maestro macro that performs the actual cuts in Blender. It lives in `blender-km-macros/`, which has the full setup guide — importing the `.kmmacros` files into Keyboard Maestro and the safety / TEST flow.

Once the macros are imported, trigger the cutting macro with the loop count that step 4 printed:

```
./blender-km-macros/scripts/trigger.sh <N>           # N =
KM loop count from step 4
./blender-km-macros/scripts/trigger.sh --test <N>    # dry
run – deletes nothing
```

trigger.sh won't fire if Blender isn't running. See the subfolder README for the macro internals and import notes.

5 — vse-remove-markers.py

Opens a .blend file in Blender headlessly, removes every timeline marker, and saves in place. Run this after the Keyboard Maestro macro has finished cutting.

```
python3 vse-remove-markers.py /path/to/project.blend
python3 vse-remove-markers.py          # prompted –
supports drag-and-drop
```

Folder Structure (after full run)

```
project_folder/
├── _ingest/
│   ├── manifest.json           ← ingest output
│   ├── clips_ordered.txt
│   ├── ingest_report.md
│   └── transcoded/
│       ├── manifest_transcoded.json ← transcode output
│       └── *.mp4                ← normalized clips
├── my_project.blend           ← VSE project
├── my_project_cut.blend       ← copy with cuts
└── applied (post-KM)
    └── blender_import.log
```

Typical Full Run

```
# 1. Ingest (answer the audio-pairing prompt if you have
Voice Memos)
python3 ingest.py ~/footage/trip

# 2. Transcode (can take a while – grab a coffee)
#     Clips paired with external audio get synced + audio-
swapped here
python3 transcode.py ~/footage/trip

# 3. Import into Blender
python3 import-vse.py ~/footage/trip --name trip_edit

# --- Open trip_edit.blend, do manual editing, place F/u
markers ---

# 4. Validate markers
python3 vse-validate-markers.py
~/footage/trip/trip_edit.blend

# 4.5 Run the Keyboard Maestro cutting macro (N = loop
count printed by step 4)
./blender-km-macros/scripts/trigger.sh <N>

# 5. Clean up markers
python3 vse-remove-markers.py
~/footage/trip/trip_edit_cut.blend
```

Platform Notes

- **macOS** is the primary target. `avconvert` (iPhone HDR → SDR) is macOS-only; on Linux, the pipeline falls back to `ffmpeg`'s `zscale` tone-map automatically.

- All scripts support **drag-and-drop** path input when run without arguments — useful in Terminal on macOS.
- Blender is auto-detected at `/Applications/Blender.app` (macOS) and `/usr/bin/blender` (Linux). Pass `--blender /path/to/blender` to override.

Roadmap

- **Proxy generation** — re-add `redo_proxies.py` as a post-import step; build 25% proxies for smooth VSE playback without leaving Blender
- **Subtitles** — auto-generate or import SRT / VTT and burn or soft-attach to the Blender timeline
- **After Effects export** — convert the VSE timeline to an AE-compatible project file (via ExtendScript or `aescript` bridge) for finishing in After Effects
- **External audio format validation** — iPhone Voice Memos are sometimes encoded at a sample rate or bitrate that causes audio drift against 29.97 CFR video; add a pre-sync check that validates the audio file's sample rate and codec before `audio-offset-finder` runs and warns (or auto-converts) if the format is likely to cause slipping
- **Ebook** — keep this PDF/ebook (Kindle edition) updated as the pipeline evolves

Part II - What It Is & Why It Exists

You shoot an event on whatever is in your hands: an iPhone, a GoPro, an older Canon Vixia camcorder, a Canon 7D, a little iVue Rincon action cam, plus a handful of still photos. You end up with a folder full of clips that disagree with each other on almost everything that matters to an editor:

- **They disagree on time.** iPhone and GoPro stamp every clip in true UTC. The Canons and the iVue write *local wall-clock* time but mislabel it as *Z* (UTC). Sorted naively, the camcorder clips land hours away from the phone clips, so your timeline is shuffled out of real order.
- **They disagree on color.** iPhone video is HDR (high dynamic range). Dropped onto an SDR timeline next to GoPro footage it looks washed-out or wrong.
- **They disagree on frame timing.** Phones often record VFR (variable frame rate) to save space; an editor wants CFR (constant frame rate) or playback drifts and audio slips.
- **They disagree on shape.** A vertical phone clip on a 16:9 timeline leaves ugly black bars, or gets stretched.

- **They disagree on being video at all.** Stills are not clips; they have no duration and cannot sit on a video track as-is.
- **They disagree on audio quality.** A clip shot with a camera's built-in mic may have poor audio when a dedicated recorder or iPhone Voice Memo was running nearby — but those two recordings need to be synced and swapped into the video.

The Video Intake Pipeline exists to erase every one of those disagreements *before* you start editing. You point it at the messy folder; it hands back a single Blender Video Sequence Editor (VSE) project where every clip is the same flavor — SDR, constant 29.97 fps, 16:9, normalized audio — laid end to end in **true chronological order**, ready to cut. The goal is simple: *spend your energy editing, not wrangling formats.*

Part III - How It Is Structured

The pipeline is not one big program. It is **five small command-line tools** that hand work to each other through plain files on disk. Think of it as an assembly line where the conveyor belt is a JSON file.

```
ingest.py      --> manifest.json
transcode.py   --> manifest_transcoded.json (+ normalized
.mp4 clips)
import-vse.py  --> project.blend
                [ you edit in Blender, place F/u markers ]
vse-validate-markers.py --> project_cut.blend (+ a "loop
count")
                [ Keyboard Maestro macro does the cutting ]
vse-remove-markers.py   --> clean project_cut.blend
```

Everything a downstream tool needs is written by the upstream tool into the `_ingest/` folder. The manifest is the **contract** between stages: ingest describes every file (where it is, when it was shot, whether it is HDR/VFR/vertical, whether it has an external audio file to sync); transcode reads that description, normalizes the media, and rewrites the manifest with the new file paths and corrected flags; import reads the final manifest and never has to re-sort anything because the order is already baked in.

Patterns shared by every script

Once you have read one of these scripts, you have half-read the others. They share a small set of idioms on purpose:

- **Drag-and-drop friendly.** Run any script with no argument and it prompts you to drag a folder (or `.blend`) into the Terminal. A `sanitize_path()` helper strips the quotes and backslash-escapes that macOS adds when you drop a path.
- **Tool auto-discovery.** Each script hunts for `ffmpeg`, `blender`, and friends on your `PATH` first, then at well-known macOS/Linux locations, and only then gives up with an install hint. This is what makes the same code run on a Mac and on Linux Mint.
- **Headless Blender by code generation.** The three Blender scripts never open a window. They *write a small Python program into a temporary file*, launch `blender --background --python that_file.py`, and watch Blender's output for a sentinel line (`IMPORT_COMPLETE`, `VALIDATION_COMPLETE`, `REMOVE_COMPLETE`). Seeing the sentinel is how the wrapper knows the job truly finished, not just that Blender exited.
- **Idempotent re-runs.** Transcode skips any clip whose output already exists, so re-running after a crash resumes instead of

redoing everything. Ingest prunes its own output folder from the scan so it never re-ingests its own transcoded copies.

- **Colored, streaming output.** Helpers like `ok()`, `warn()`, `skip()` print green / yellow / purple status as work happens, and Blender's own log lines are echoed live with a `blender |` prefix so a long import never looks frozen.

Part IV - How Each Stage Works

Stage 1 -- ingest.py: build the map

Ingest walks the footage folder, inspects every video and image, and writes the manifest plus two human-readable reports. The interesting work is in how it figures things out without trusting any single piece of metadata.

Source detection. It guesses the camera with a cascade: GoPro filename prefixes (GH , GP , GOPR , GL), then ffprobe tags (an Apple make tag means iPhone; a GoPro encoder/handler means GoPro), then iPhone naming patterns (IMG_1234 , screen recordings). Anything unrecognized is simply unknown — it still flows through, it just is not special-cased.

The timezone fix (the clever part). This is the pipeline's signature trick. A clip only counts as "really UTC" if it carries proof: an Apple make tag or Apple creationdate , or a GoPro signature. iPhone and GoPro pass; the Canon Vixia, Canon 7D, and iVue Rincon do not — they record local time mislabeled as UTC. Ingest:

1. Collects the true UTC offset from any iPhone/GoPro clip in the batch (it reads the offset baked into Apple's `creationdate`, e.g. `-0400`, and takes the most common one).
2. Rewrites every "naive-local" clip from that one batch offset into true UTC, so the whole shoot finally shares a single clock.
3. If there is no iPhone/GoPro clip to learn the offset from, it falls back to *this machine's* timezone and prints a warning telling you to re-run with `--local-offset` if the footage was shot elsewhere.

Crucially it **ignores any timezone tag the camera itself embedded**, because those are routinely wrong (the Vixia happily reports `-05:00` while shooting in `-04:00`, which would re-break the order). Trust the phones; correct the rest.

External audio pairing. After the scan completes, ingest asks whether any video clips have a separate higher-quality audio file — an iPhone Voice Memo, a dedicated recorder, anything with better audio than the camera's built-in mic. If you say yes, it walks you through drag-and-dropping each video clip and its matching audio file one pair at a time. No processing happens at this point; the pairing is simply written into `manifest.json` as an `external_audio` field on the relevant clip entry. `Transcode.py` picks it up in step 2.

Other detectives. Rotation is read from the modern iPhone location — the Display Matrix in the stream's `side_data_list`, not the legacy `rotate` tag — and width/height are swapped when a clip is 90/270 degrees so orientation is logical, not pixel-literal. HDR is detected from the color transfer curve (`smpte2084` for PQ, `arib-std-b67` / `hlg` for HLG). VFR is inferred when a clip's average and "real" frame rates disagree by more than half a frame. Stills with no usable `ffprobe` data fall back to `ImageMagick identify` for size and `exiftool` for EXIF orientation and timestamp.

What you get. `manifest.json` (the machine contract), `clips_ordered.txt` (a quick chronological table), and `ingest_report.md` — a narrative with per-camera counts, total duration, flagged clips (HDR/VFR/vertical/no-timestamp), and any **timeline gap longer than 10 minutes**, which is a handy way to see where one scene ended and the next began.

Stage 2 -- `transcode.py`: make everything agree

Transcode reads the manifest and converts each file according to what it is. The whole point is that every output is the same: SDR, CFR 29.97, 16:9, libx264 MP4 with audio normalized to **-16 LUFS / -1.5 dBTP**.

- **Stills become 6-second clips.** A three-pass recipe: ImageMagick pads the photo onto a 4K black canvas; ffmpeg makes a 1-second static lead-in; ffmpeg makes a 5-second slow "Ken Burns" zoom (`zoompan` , easing from 1.0 to 1.1x); the two are concatenated. The result drops onto the timeline like any clip.
- **iPhone HDR uses Apple's own converter.** macOS `avconvert` with the `PresetAppleM4V1080pHD` preset does the HDR-to-SDR tone-map (Apple knows its own color science best), then ffmpeg handles any leftover VFR-to-CFR or vertical-to-16:9 work. iPhone clips are *always* forced to CFR 29.97.
- **Non-Apple HDR uses an ffmpeg tone-map** — a `zscale -> tonemap=hable -> zscale` chain that linearizes, maps highlights with the Hable curve, and lands back in bt709 SDR. (On Linux, where there is no `avconvert` , *all* HDR takes this path.)
- **Vertical clips get a blurred 16:9 letterbox.** The clip is split into two copies; one is scaled up to fill 1920x1080, heavily Gaussian-blurred (`gblur=sigma=120`) and darkened to make a soft background; the original portrait is centered on top. Far nicer than black bars.
- **Clean GoPro / other clips are stream-copied** — no re-encode, so they are fast and lossless to pass through.
- **Clips with external audio get synced and swapped.** When a manifest entry has an `external_audio` field (set during ingest),

transcode runs `audio-offset-finder` to measure the time offset between the clip's native audio and the external file, then uses `ffmpeg` to replace the native audio track with the synced external recording. All other transforms (HDR, VFR, vertical) still apply. A confidence score is printed; a score below 10 triggers a warning that the sync may be inaccurate.

Re-encoded video uses CRF 18 (visually lossless) at the fast preset. After each file, transcode re-probes the *output* to record its true dimensions and clear the HDR/VFR flags, then writes `manifest_transcoded.json`. Because it skips any output that already exists, the step is safe to re-run.

Stage 3 -- `import-vse.py`: build the Blender project

This stage turns the normalized clips into an actual `.blend`. It detects the frame rate from the first clip, asks you for a project name and resolution (1080p or 4K), shows you the import order for confirmation, then **generates a Blender Python script** and runs it headlessly.

Inside Blender, the generated script sets the scene resolution and fps, finds or creates a Sequence Editor area, and adds each clip with `movie_strip_add` at the current playhead — then advances the playhead to that strip's end, so clips land back-to-back in order with

no manual math. It selects all strips, loads Blender's built-in **Video Editing** workspace so the file opens with the right layout, saves the `.blend`, and prints `IMPORT_COMPLETE`. A `blender_import.log` records exactly what was placed and when.

Stage 4 -- `vse-validate-markers.py`: check your cut plan

After you edit in Blender, you mark the ranges to **keep** with timeline markers following one convention: the start of a keep-range is named `F1`, `F2`, ... and its end is named exactly `u`. Each `F.. -> u` pair is one segment to retain.

Validation opens the file headlessly, sorts the markers by frame, and checks: the count is even (every start has an end), the names alternate `F..` then `u`, and no two keep-ranges overlap (a pairwise comparison). If anything is wrong it lists every problem and exits without touching your file. If everything is clean it saves a `_cut.blend` copy and prints the **loop count** for the Keyboard Maestro macro — which is `pairs - 1`, the number of unwanted gaps *between* your kept segments that the macro will delete.

Stage 4.5 -- the Keyboard Maestro macro: do the cutting

Blender has no clean headless "ripple delete between markers" operation, so the actual cutting is done by simulating the keystrokes a human would press — and that is what Keyboard Maestro automates. The macro lives in `blender-km-macros/` and is fired by `trigger.sh`:

```
./blender-km-macros/scripts/trigger.sh <N>          # N =  
loop count from Stage 4  
./blender-km-macros/scripts/trigger.sh --test <N>    #  
rehearse, delete nothing
```

`trigger.sh` first checks Blender is actually running (`pgrep`), then uses `osascript` to tell the Keyboard Maestro Engine to run the macro, passing `N` as a parameter. Inside the macro that value arrives as `%TriggerValue%`, gets stored in a variable `BlenderDeleteCount`, and drives a Repeat-N-times block of the fixed delete keystrokes. There is a (TEST) twin that runs the same flow but pops a confirmation dialog instead of deleting, so you can rehearse safely.

Stage 5 -- vse-remove-markers.py: tidy up

Once the macro has done its cutting, the `F..u` markers are just clutter. This last script opens the `.blend` file, removes every

timeline marker, and saves in place — leaving a clean, edit-ready project.

Part V - How It Was Made: Design Decisions

A few choices are worth understanding, because they explain *why* the code looks the way it does.

- **The manifest is a contract, and you can edit it.** Because stages talk through `manifest.json`, you can open it and delete a clip, fix a timestamp, or reorder by hand, and the rest of the pipeline obeys. The format is the API.
- **Generated Blender scripts plus a sentinel string.** Rather than fight Blender's Python console, each Blender stage writes a one-shot script to a temp file and watches stdout for a unique "I finished" line. It is a simple, robust way to drive a giant GUI app from a tiny CLI wrapper, and the same trick appears three times.
- **Trust phones, correct camcorders.** The timezone normalization deliberately distrusts camera-embedded timezone tags and instead anchors the whole batch to an iPhone/GoPro clock. The most recent relevant commit in the repo is: *"ingest: normalize naive-local camera timestamps to UTC."*

- **Ingest records intent, transcode does work.** The external audio pairing is a deliberate split: ingest is fast (metadata only), so asking about audio pairs there costs nothing. The heavy lifting — running `audio-offset-finder` and re-encoding with `ffmpeg` — belongs in transcode, which already owns the slow per-file processing loop. The manifest `external_audio` field is the handoff.
- **Idempotency as a habit.** Skipping existing outputs and pruning the output folder from scans both come from the same instinct: re-running should be cheap and safe, never destructive.
- **Loudness target of -16 LUFS / -1.5 dBTP.** A sensible web/streaming loudness with about 1.5 dB of true-peak headroom, so mixed-source audio sits at a consistent level instead of lurching between a quiet camcorder and a loud phone.
- **A quirk preserved in the macro docs.** Keyboard Maestro's Repeat count must be the *bare* variable name `BlenderDeleteCount` (no percent signs), because that field is a calculation, not text. The README calls this out because it trips people up every time.
- **Two repos became one, with history.** The Keyboard Maestro tool started life as its own 8-commit git repository. It was folded into this repo with `git subtree` so all eight of its original

commits are preserved in the combined history — not flattened away — and it now reads as pipeline "step 4.5".

Part VI - How to Use It: Full Walkthrough

Install the dependencies (macOS): `brew install ffmpeg imagemagick exiftool`, install Blender, and install Keyboard Maestro for the cutting step. `avconvert` is already built into macOS. If you plan to use external audio files: `pip install audio-offset-finder`.

1. Ingest. Scan the footage and read the report:

```
python3 ingest.py ~/footage/trip
```

The script scans and reports. Then it asks:

```
Any of these clips have a separate external audio file?  
[y/N]:
```

If you recorded a Voice Memo or used a separate recorder alongside any clip, answer `y` and drag-and-drop each video + audio file pair when prompted. Otherwise press `Return` to skip. Open `~/footage/trip/_ingest/ingest_report.md` and skim the flags. If a camera with no phone clip in the batch got the wrong time, re-run with the real offset:


```
python3 ingest.py ~/footage/trip --local-offset -04:00
```

2. **Transcode.** Normalize everything (this is the slow step):

```
python3 transcode.py ~/footage/trip
```

Clips with paired external audio are synced and audio-swapped here automatically. Use `--dry-run` first if you want to see the plan, or `--no-loudnorm` to skip audio leveling.

3. **Import into Blender.** Build the project:

```
python3 import-vse.py ~/footage/trip --name trip_edit
```

Pick a resolution, confirm the clip order, and let Blender build `trip_edit.blend`.

4. Edit and mark. Open the `.blend`, do your rough edit, and place `F1.../u` marker pairs around every range you want to keep. Then validate:

```
python3 vse-validate-markers.py  
~/footage/trip/trip_edit.blend
```

Note the printed **loop count** and the new `trip_edit_cut.blend`.

4.5. Cut. With Blender open and the mouse hovering over the VSE timeline, run the macro with that loop count:

```
./blender-km-macros/scripts/trigger.sh <loop-count>
```

(Rehearse with `--test <loop-count>` first if you like.)

5. **Clean up.** Remove the now-spent markers:

```
python3 vse-remove-markers.py  
~/footage/trip/trip_edit_cut.blend
```

That is the whole loop: a folder of mismatched footage in, a clean cut Blender project out.

Troubleshooting quick hits

- *Clips out of order by hours* — a camcorder's time was not corrected; re-run ingest with `--local-offset`.
- *A file sorted to the very end* — it had no timestamp metadata; the report flags these.
- *Import found no manifest* — you skipped a step; run ingest then transcode first.
- *Macro does nothing* — Blender must be running and the mouse must be over the timeline; `trigger.sh` will warn if Blender is closed.
- *Audio sync looks off* — check the `standard_score` printed during transcode; a score below 10 means low confidence. iPhone Voice

Memos in particular can have sample-rate quirks that cause drift
— a future validation step is on the roadmap.

Part VII - Glossary

- **HDR / SDR** -- High / Standard Dynamic Range. HDR clips (iPhone) carry a wider brightness range and must be tone-mapped down to SDR for a normal timeline.
- **PQ / HLG** -- the two HDR transfer curves (`smpte2084` and `arib-std-b67`); their presence is how ingest detects HDR.
- **VFR / CFR** -- Variable / Constant Frame Rate. Phones shoot VFR; editors want CFR (here, 29.97 fps) so audio and video stay in sync.
- **FPS / 29.97** -- frames per second. 29.97 (technically 30000/1001) is the pipeline's target timeline rate.
- **LUFS / dBTP** -- Loudness Units Full Scale / decibels True Peak. The audio targets: -16 LUFS average loudness, -1.5 dBTP peak ceiling.
- **loudnorm** -- the ffmpeg filter that measures and adjusts loudness to those targets.
- **Ken Burns** -- the slow pan/zoom applied to still photos so they feel alive.

- **VSE** -- Blender's Video Sequence Editor, the timeline the project is built in.
- **zscale / tonemap (Hable)** -- the ffmpeg color tools used for non-Apple HDR-to-SDR conversion; "Hable" is a filmic highlight-rolloff curve.
- **letterbox** -- fitting non-16:9 content onto a 16:9 frame; here a blurred copy of the clip fills the sides instead of black bars.
- **audio-offset-finder** -- the Python tool (`pip install audio-offset-finder`) that compares two audio tracks and returns the time offset between them, used to sync external audio recordings to their matching video clips.
- **external_audio** -- a field written into `manifest.json` by ingest when you pair a video clip with a separate audio file; transcode reads it to know which clips need audio replacement.
- **manifest** -- the JSON file describing every clip; the contract between stages.
- **idempotent** -- safe to run again: re-running skips work already done.
- **headless / --background** -- running Blender with no window, driven entirely by a script.
- **ffprobe / exiftool / identify** -- the inspectors: media streams, EXIF tags, and image dimensions respectively.

- **git subtree** -- the merge technique that folded the macro repo into this one while keeping its commit history.
- **percent-TriggerValue-percent** -- the Keyboard Maestro token holding the value passed to a macro; here, the number of delete loops.

Appendix A - The blender-km-macros

README (verbatim)

blender-km-macros

A small Keyboard Maestro macro manager for a Blender clip-deletion workflow. It bundles the exported macros plus a shell helper so the "Delete clips Blender" macro can be triggered from anywhere (terminal, scripts, other apps), running the clip-deletion cycle a number of times you pass in.

This folder is **part of the video-intake-pipeline repository** — it provides **step 4.5** of that pipeline: the clip-cutting macro you run after `vse-validate-markers.py` and before `vse-remove-markers.py`. See the top-level README for the full footage → edit workflow.

Layout within the repo

```

video-intake-pipeline/                ← repo root (ingest.py,
transcode.py, ...)
└─ blender-km-macros/
    │─ README.md
    │─ macros/
    │   │─ Delete clips Blender.kmmacros          # the
real macro
    │   └─ Delete clips Blender (TEST).kmmacros    # dry-
run version (no deletions)
    └─ scripts/
        └─ trigger.sh                            # CLI
wrapper around the osascript trigger

```

Importing the macros

Double-click each .kmmacros file to import it into Keyboard Maestro (or drag it onto the Keyboard Maestro Editor). Both import into a macro group named **Blender group**.

Group availability: the exported group is set to be available in **all applications** (not restricted to Blender). This is required — the macro brings Blender to the front itself, so it must be allowed to run even when Blender isn't the front app. After importing, confirm the Blender group shows "available in all applications."

Using trigger.sh

scripts/trigger.sh triggers a macro and passes a number through as the macro's parameter. That number is how many times the macro repeats the clip-deletion cycle.


```
./scripts/trigger.sh 5          # run the REAL macro, 5  
delete cycles  
./scripts/trigger.sh --test 5    # run the TEST macro (no  
deletions), see below  
./scripts/trigger.sh -t 5       # same as --test
```

Paths above are relative to this `blender-km-macros/` folder. From the repo root, prefix with the folder name — e.g. `./blender-km-macros/scripts/trigger.sh 5` — which is the form the top-level README uses for step 4.5.

- Requires a number argument (after the optional `-t / --test` flag).
- Exits with an error and a usage message if the number is missing.
- **If Blender isn't running, it does not trigger the macro** — it prints a message to stderr, posts a macOS notification, and exits 1. (The KM gate is still a backstop for other trigger paths.)
- The script is executable; if needed, restore the bit with `chmod +x scripts/trigger.sh`.

Manual triggering with `osascript`

The same triggers without the wrapper script — substitute your count for 5:

```
osascript -e 'tell application "Keyboard Maestro Engine" to
do script "Delete clips Blender" with parameter "5"'
osascript -e 'tell application "Keyboard Maestro Engine" to
do script "Delete clips Blender (TEST)" with parameter "5"'
```

The macros read the passed value via the `%TriggerValue%` token.

How the macro works

Delete clips Blender is a single, self-contained macro in the **Blender group**. When triggered with parameter `N`, it runs this sequence:

1. **Gate — only if Blender is open.** If Blender is *not* running, it shows a message and cancels. Nothing else happens.
2. **Bring Blender to the front.** Activates Blender so the simulated keystrokes are guaranteed to land in Blender and not whatever app you triggered from.
3. **READY? prompt.** A prompt tells you to move the mouse over the Blender VSE timeline (required — those shortcuts only work when the mouse is over the timeline), then **press Return to start** or **Escape to cancel**.
4. **Capture the count.** A **Set Variable to Text** action stores the passed parameter (`%TriggerValue%`) into the variable `BlenderDeleteCount`.

5. **Repeat the delete cycle N times.** A **Repeat** action whose count is the calculation `BlenderDeleteCount` runs the fixed clip-deletion keystroke sequence (with 1-second pauses) N times. The count is driven entirely by the argument passed from `trigger.sh / osascript` — there is no hard-coded count.

The Repeat count must be the **bare variable name**

`BlenderDeleteCount` (no % signs) — that field is a calculation, not text. Putting `%TriggerValue%` or `%BlenderDeleteCount%` there shows "invalid." The editor may briefly show "invalid" until the variable has a value; that's just a preview and it works at run time (the Set Variable action runs first).

Testing it safely (the TEST macro)

Delete clips Blender (TEST) runs the same front-half flow — gate, bring-to-front, READY? prompt — but **replaces the delete keystrokes with a confirmation dialog** instead of actually deleting anything. Use it to confirm the flow works end to end (including that the count is passed through correctly) before running the real thing:

```
./scripts/trigger.sh --test 5
```

After you press Return (Start) at the READY? prompt, you'll get a "  **TEST PASSED** " dialog that echoes the count it received (5), with an OK button. No clips are touched.

Notes

- These files are **exported from a working Keyboard Maestro setup** (with one action — the TEST macro's " TEST PASSED" confirmation — appended afterward), so they should import cleanly. The TEST macro's gate cancels silently if Blender isn't open (its "Blender isn't open" message did not survive an earlier import); add a message there in the editor if you want one.
- A bare hotkey is intentionally *not* set: the count comes from the passed parameter, so the macros are triggered by name (via `trigger.sh`, `osascript`, or an *Execute a Macro* action), not a keypress.
- The **Blender group** is available in all applications (required so the macro can bring Blender to the front from anywhere).

After any edit in Keyboard Maestro, **re-export the macro over the matching file in macros/** to keep the macros in sync — that round-trip is the source of truth, not hand-edited XML. Commit the re-exported `.kmmacros` in the combined video-intake-pipeline repo; this folder no longer has its own git history — it was merged in via `git subtree`.